CHMLTECH ONLINE JOURNAL · VOLUME 105, ISSUE 4 · RESEARCH ARTICLE

Agent Interoperability Standards: The Model Context Protocol, Agent-to-Agent Protocols and Integration with ERP, CRM and Plant Systems

Quak Foo Lee

Chief Technology Officer, Chmltech Ltd.

Division of Artificial Intelligence and Machine Learning · Vancouver, BC, Canada

Published November 12, 2026 14 min read

ABSTRACT

Integration with existing systems is now the most common barrier to deploying AI agents: in a 2025 survey of more than 500 technical leaders, 46% named it their primary challenge, ahead of data access and quality at 42%. This paper examines the open standards that aim to lower that barrier. The Model Context Protocol (MCP) standardizes how an agent discovers and calls tools and data; its 2026-07-28 revision made the protocol stateless, added an extensions framework and enterprise-managed authorization, and the protocol is now governed by the Linux Foundation's Agentic AI Foundation. The Agent2Agent (A2A) protocol, which reached a stable version 1.0 in 2026, standardizes how independent agents discover each other and delegate tasks. The paper explains the architecture of both, shows why a standard interface turns an N × M connector problem into an N + M one, and sets out a reference architecture for connecting agents to ERP, CRM and plant systems (manufacturing execution systems, historians and OPC UA servers) through a governed MCP gateway. It closes with design rules for staged writes, operational-technology zoning and protocol

versioning, a worked downtime investigation, and a staged adoption path for mid-sized industrial firms. The central conclusion is that the standards remove much of the connector burden, but the value and the safety come from the architecture built around them.

Keywords: Model Context Protocol; Agent2Agent; agent interoperability; ERP integration; OPC UA; manufacturing execution systems; enterprise architecture

1. Introduction

An AI agent is only as useful as the systems it can reach. A purchasing agent that cannot read open purchase orders in the ERP, or a maintenance agent that cannot see vibration trends in the plant historian, produces advice rather than work. For most firms the model is no longer the constraint. In a survey of more than 500 technical leaders published in December 2025, integration with existing systems was the most cited deployment challenge (46%), followed by data access and quality (42%) and change management (39%) [1]. The same survey found that 57% of organizations already run agents on multi-stage workflows, which multiplies the number of systems each agent must touch [1].

The cost of getting integration wrong is visible in the failure forecasts. Gartner expects more than 40% of agentic AI projects to be cancelled by the end of 2027, citing escalating costs, unclear business value and inadequate risk controls [2]. Gartner also advises against bolting agents onto legacy systems simply to tick a box [3]. Integration debt is one of the main ways a promising pilot becomes a cancelled project.

This paper is the first in a short Chmltech series on agentic AI in industrial and mid-market firms. It covers:

1. Why point-to-point agent integration does not scale (Section 2).
2. The Model Context Protocol: architecture, primitives and the 2026-07-28 revision (Section 3).
3. Agent-to-agent protocols, chiefly A2A, and how they complement MCP (Section 4).
4. A reference architecture for ERP, CRM and plant-system integration (Section 5).
5. Design rules, risks and a staged adoption path (Section 6).

Security, identity and audit are treated only where they shape the integration architecture; they are the subject of the companion paper on agent security and identity in this volume.

2. The Integration Problem

Enterprise application estates are large and loosely connected. MuleSoft's 2025 Connectivity Benchmark, a survey of 1,050 IT leaders, found that organizations run an average of 897 applications, with only 29% of them typically connected; firms already using agents ran even more applications, 1,103 on average [4]. Each agent that needs to act across this estate inherits the gaps.

2.1 From N × M to N + M

Before a common protocol, each agent framework needed its own connector to each system. With N agent hosts and M target systems, the number of custom connectors to build and maintain is given by Eq. (1):

$$C_{\text{custom}} = N \times M \tag{1}$$

A standard interface lets each agent host implement one client and each system expose one server, so the count becomes Eq. (2):

$$C_{\text{standard}} = N + M \tag{2}$$

Worked example (illustrative figures). A mid-sized manufacturer plans four agents (a desktop assistant, a maintenance agent, a scheduling agent and a customer-service agent) over six systems (ERP, CRM, MES, historian, CMMS and QMS). Point-to-point integration needs 4 × 6 = 24 connectors; a standard interface needs 4 + 6 = 10 adapters, a 58% reduction. The gap widens with every agent or system added, and it matters more for maintenance than for the first build, because a custom connector breaks whenever either side changes its API.

2.2 Why agents make integration harder

Agents differ from conventional integration clients in three ways that standard interfaces must address:

- **Discovery at run time.** An agent chooses tools while reasoning, so it needs machine-readable descriptions of what each tool does, its inputs and its side effects, not just an endpoint.
- **Delegated authority.** An agent acts for a person or a team, so every call must carry whom it acts for and what it may do. A shared service account hides that.
- **Long-running work.** A batch reconciliation or a production reschedule can take minutes to hours, which a simple request-response interface handles poorly.

The protocols in Sections 3 and 4 are best read as answers to these three needs.

3. The Model Context Protocol

MCP is an open protocol, first released by Anthropic in November 2024, that defines how AI applications connect to external tools, data and other applications in a client-server arrangement [5]. In December 2025 Anthropic donated it to the Agentic AI Foundation (AAIF), a new Linux Foundation body launched alongside OpenAI's AGENTS.md and Block's goose, which placed the protocol under vendor-neutral governance [6].

3.1 Architecture and primitives

An MCP **host** (a chat application, a development environment or an agent runtime) runs one **client** per connection; each client talks to an MCP **server** that wraps a system such as an ERP interface or a historian. Messages use JSON-RPC 2.0. Local servers typically run over standard input and output; remote servers run over HTTP [7]. Servers expose three main primitives, summarized in Table 1.

Table 1. MCP server primitives, with industrial examples.

Primitive	What it exposes	Industrial example
Tools	Functions the model may call, with a JSON schema for inputs	get_open_work_orders(asset_id) on the CMMS
Resources	Read-only data addressed by URI	The current SOP for a reactor start-up
Prompts	Reusable, parameterized templates	A shift-handover summary template

Source: MCP specification [7]; examples by the author.

The tool description is the contract the agent reasons over, so its wording, input schema and stated side effects decide whether the agent uses the tool correctly. Poorly described tools are a common source of wrong calls.

3.2 The 2026-07-28 revision

The July 2026 revision is the largest change since remote MCP launched and was aimed squarely at enterprise operation [5, 8]:

- **Stateless requests.** The initialize handshake and the protocol-level session are gone. Each request carries its protocol version and the client capabilities the server needs, so servers can sit behind ordinary load balancers and scale on existing Kubernetes tooling without sticky routing [5, 8].
- **Extensions framework.** Optional capabilities are now formal extensions that are negotiated and versioned separately from the core protocol [8].
- **Tasks as an extension.** Long-running tool calls become asynchronous tasks with durable handles, so a client can resume polling after a crash instead of holding a connection open [5].
- **Enterprise-managed authorization.** An extension lets organizations govern access to MCP servers through their central identity provider; the revision also asks for the issuer (iss) parameter to be included and validated in authorization responses, which defends against OAuth mix-up attacks when one client talks to several servers [5].
- **Deprecation policy.** Features now move through a published lifecycle with at least 12 months between deprecation and removal, which gives implementers a planning horizon [5].

For integrators the practical consequence is that MCP servers can now be run like any other stateless web service. Teams that built servers on session identifiers face a migration [5].

3.3 Security model in brief

MCP authorization is based on OAuth. The specification's security guidance forbids token passthrough: a server must not accept tokens that were not issued for it, and must not forward a client's token to a downstream interface, because doing so bypasses audience checks, rate limits and audit trails [9]. The design rule for integrators is that the MCP server

obtains its own, narrowly scoped credential for each downstream system. The companion security paper treats this in depth.

3.4 Writing tool descriptions agents use correctly

Because the model chooses tools from their descriptions, a tool definition is a specification, not a comment. A good definition states what the tool does, when not to use it, the units and allowed ranges of each input, and its side effects. Table 2 shows the fields of a read-only historian query for one asset.

Table 2. Example definition of a read-only historian tool (illustrative).

Field	Value
name	get_tag_trend
description	Read-only. Returns time-stamped values of one historian tag for one asset. Use for trend and downtime analysis. Does not change any setpoint. Maximum window 7 days.
asset_id (string, required)	Asset number from the CMMS, for example P-2301
tag (string, required)	One of: flow, pressure, temperature, vibration
start, end (date-time, required)	Time window to return

Author's example, following the tool structure in the MCP specification [7].

Three features of the example generalize. The first word of the description states the risk class. An enumeration replaces free-text tag names, so the model cannot request tags outside the curated set. And the stated limit of seven days is enforced again on the server, because a description guides the model but does not bind it.

4. Agent-to-Agent Protocols

MCP connects an agent to tools. It does not define how two independent agents, built by different vendors and running under different owners, find each other and share work. The

Agent2Agent (A2A) protocol fills that gap. Google developed it and contributed it to the Linux Foundation; in April 2026 the project reported more than 150 supporting organizations, a stable version 1.0 specification and integration into the major cloud platforms [10]. The Linux Foundation describes A2A and MCP as complementary [10].

4.1 Core concepts

- **Agent card.** Each agent publishes a machine-readable description of its skills, its endpoint and the authentication it requires, so another agent can decide whether and how to call it.
- **Task.** The unit of delegated work, with a lifecycle (submitted, working, needing input, completed, failed) that both sides can track.
- **Messages and artifacts.** Agents exchange messages while a task runs and return artifacts (a document, a data set, a structured result) when it ends; long tasks can stream updates.

The key design choice is opacity: an A2A caller sees another agent's declared skills and results, not its internal tools, prompts or memory. That suits work crossing an organizational or vendor boundary, where each party keeps control of its own systems.

4.2 Choosing the right protocol

Table 3 compares the two agent protocols with OPC UA, the established interface to industrial control systems.

Table 3. MCP, A2A and OPC UA compared.

Question	MCP	A2A	OPC UA
What connects	An agent to a tool or data source	An agent to another agent	A client to industrial devices and control systems
Unit of exchange	Tool call, resource read	Task with messages and artifacts	Node read or write, subscription, method call
Who reasons	Only the calling agent	Both agents	Neither (deterministic)
Typical industrial use	Agent reads ERP orders and historian trends	Planning agent asks a supplier's agent for a delivery date	SCADA reads tag values from a PLC
Governance	Agentic AI Foundation (Linux Foundation)	Linux Foundation A2A project	OPC Foundation

Sources: [6, 7, 10]; industrial examples by the author.

The three are layers, not rivals. In a plant, OPC UA remains the deterministic interface to control systems; an MCP server can wrap a read-only view of it for agents; and A2A lets a plant-side agent hand a task to an agent run by another firm. Figure 1 shows how the three layers combine in one cross-company exchange.

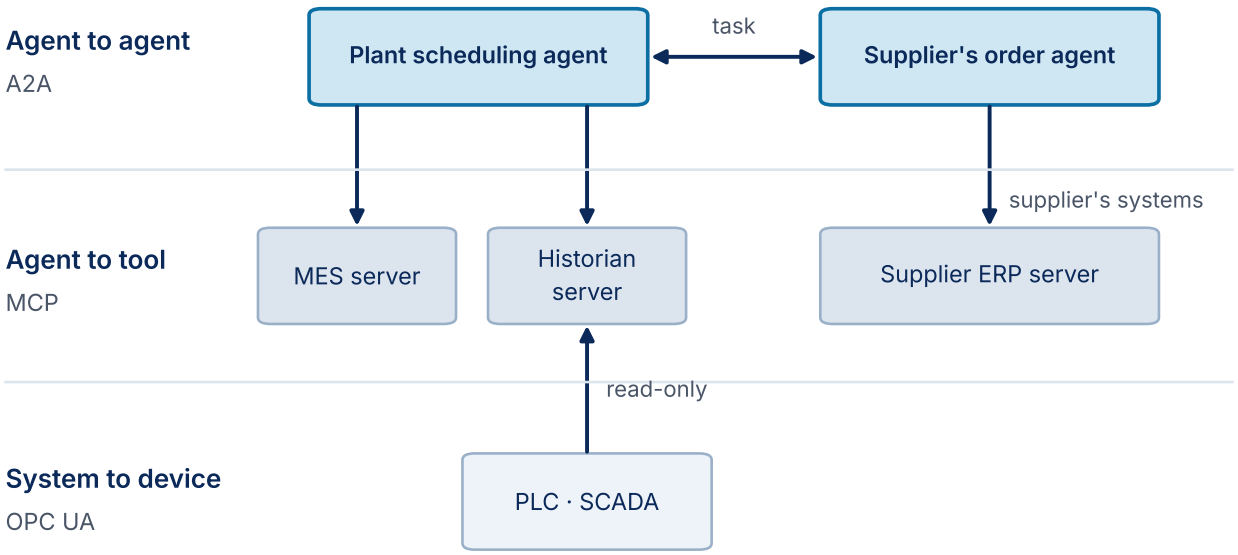


Figure 1. Protocol layers in a plant-to-supplier exchange. The plant's scheduling agent reads MES and historian data through MCP servers, the historian reads control systems over OPC UA, and the scheduling agent hands a delivery query to the supplier's agent over A2A without seeing the supplier's systems.

Drawn by the author.

4.3 Vendor uptake

Large application vendors are now building on both protocols. SAP has announced that agents built in Joule Studio will use MCP and A2A to draw on tools and third-party agents, and that bi-directional A2A will let outside agents call Joule agents inside enterprise processes; general availability of these A2A capabilities was planned for the fourth quarter of 2026 [11]. For a small or mid-sized firm on a mainstream ERP, this means agent interoperability will increasingly arrive as a vendor feature rather than a custom build, which shifts the integration task from writing connectors to governing them.

4.4 Trust between agents

A2A moves the trust question from which tools an agent may call to which outside agents it may deal with, and on what terms. Three practices keep that manageable. First, keep an allow-list of partner agents and pin each to a known endpoint and published agent card, rather than accepting any agent that presents a card. Second, authenticate each call with the method the partner's agent card declares, using credentials issued for that partnership alone.

Third, put the commercial terms outside the protocol: what a partner's agent may request, what a reply commits either firm to, and who is liable for errors belong in the supply agreement, just as they do for electronic data interchange today. An A2A reply should be treated as an input to a decision, not as a decision.

5. Integrating with ERP, CRM and Plant Systems

The recommended pattern places a single governed MCP gateway between all agent hosts and all system-facing MCP servers, and keeps the plant control layer out of reach of agents entirely (Figure 2). The layering follows the familiar ISA-95 model: business systems at Level 4, manufacturing operations (MES, historian, maintenance) at Level 3, and control systems at Level 2.

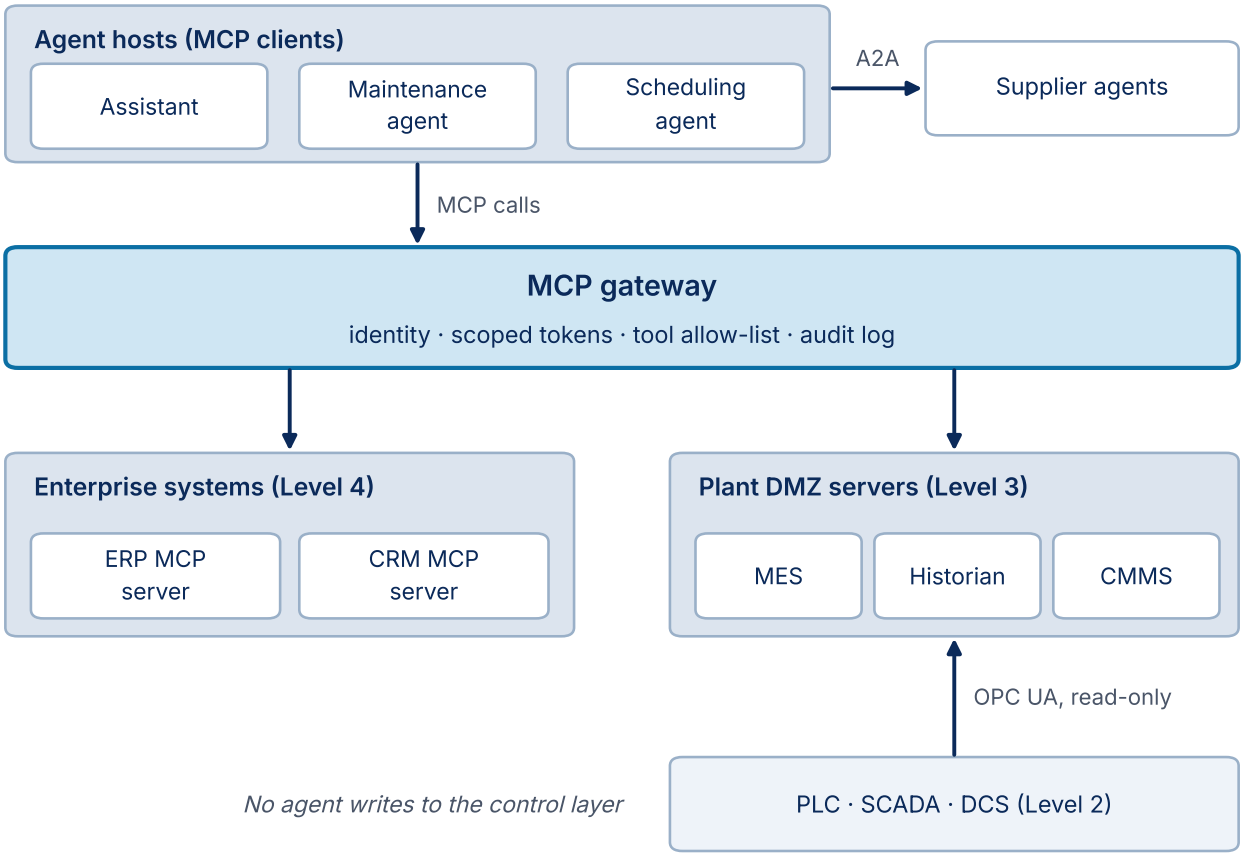


Figure 2. Reference architecture for agent integration in an industrial firm. Agents reach ERP and CRM through one governed MCP gateway; plant data reaches agents only through Level 3 servers in the industrial DMZ, which read from control systems over OPC UA.

Drawn by the author.

5.1 Enterprise systems (ERP and CRM)

ERP and CRM vendors increasingly ship their own MCP servers and A2A endpoints (Section 4.3), so the integrator's main job is to decide which tools to expose, to whom, and with what write rights. Three rules work well:

- **Expose business operations, not tables.** A tool named create_purchase_requisition_draft with a validated schema is safer and easier for the model to use than generic database access.
- **Write through drafts.** Agent writes create draft or pending records (a requisition, a quote, a credit memo) that the ERP's existing approval workflow releases. This reuses controls the business already trusts.
- **Keep the system of record authoritative.** Agents read master data from the ERP at call time rather than caching it in their own memory, so pricing, inventory and credit limits stay current.

5.2 Plant systems (MES, historian, CMMS and OPC UA)

Industrial data platforms have begun to expose plant data over MCP. HighByte's Intelligence Hub, for example, connects to OPC UA servers, historians, SCADA and MES and can present a curated, contextualized model of that data to agents through an MCP server [12]. Other platforms let agents browse OPC UA address spaces and query historians to investigate quality or downtime events [13]. Community MCP servers for OPC UA and Modbus also exist, but they vary widely in maturity and should be treated as prototypes [14]. For plant integration we recommend:

- **Read-only by default.** Agents read tags, trends, batch records and work orders. Any write (a setpoint, a recipe parameter, a mode change) stays with the control system and its operators.
- **Contextualize before exposing.** Raw tag names such as FIC-2301.PV mean little to a model. A hub that maps tags to assets, units and engineering limits gives the agent the context it needs and limits what it can see.
- **Respect the zone model.** MCP servers for plant data sit in the industrial DMZ and connect outward to the gateway; nothing in the enterprise network opens a path into the control network. This is consistent with the zones-and-conduits approach of ISA/IEC 62443.

- **Write to Level 3, not Level 2.** Where agents act on the plant, they act on operations records: raising a maintenance notification in the CMMS, proposing a schedule change in the MES, or opening a deviation in the QMS, each for a person to approve.

5.3 Integration patterns compared

Table 4 groups the ways an agent can connect to these systems by risk and by the control that contains it.

Table 4. Integration patterns for agents in an industrial firm, by risk and control.

Pattern	Agent can	Typical use	Risk	Control
Read-only query	Read records and trends	Downtime analysis, order status	Low	Tool allow-list, row-level permissions
Staged write	Create drafts for approval	Requisitions, work requests, quotes	Medium	Existing approval workflow
Event-driven	React to alarms or status changes	Maintenance triage, late-order alerts	Medium	Rate limits, deduplication
Agent-to-agent (A2A)	Delegate a task to another agent	Supplier delivery confirmation	Medium to high	Pinned agent cards, contract terms
Direct control write	Change setpoints	Not recommended	High	Keep outside agent reach

Author's classification.

5.4 Worked example: a downtime investigation

The following sequence shows the architecture in use. A production supervisor asks the maintenance agent why Line 2 lost output on the night shift.

1. The agent host authenticates the supervisor and calls the gateway as the maintenance agent acting for that supervisor.

2. The gateway checks the agent's allow-list and forwards read-only calls to the MES server (line states and stop codes for the shift) and to the historian server (get_tag_trend for the line's main pump).
3. The historian server reads the contextualized pump tags, which its hub collects from the control system over OPC UA. Nothing is written to the control layer.
4. The agent finds a stop that coincides with a vibration rise and calls the CMMS server's create_work_request_draft tool, a staged write.
5. The supervisor reviews and releases the work request in the CMMS. Every call, from the first query to the release, sits in the gateway's audit log under one trace identifier.

The agent saved the supervisor the cross-system search, but every action that changes a record still passed through a person and an existing workflow.

6. Design Guidance, Risks and Adoption Path

6.1 Design rules

1. **One gateway, many servers.** Route every agent call through a single MCP gateway that authenticates the agent, attaches the user it acts for, enforces a per-agent tool allow-list and writes the audit record. Servers then stay simple and focused on one system.
2. **Treat tool descriptions as engineering documents.** Each tool needs a clear name, a precise description, a strict input schema and a statement of side effects. Review them as carefully as a P&ID: the agent will do exactly what the description lets it believe is safe.
3. **Prefer narrow tools to broad ones.** Ten specific tools with validated inputs are easier to secure, test and audit than one tool that accepts free-form queries.
4. **Design for asynchronous work.** Use the tasks extension for reconciliations, report generation and schedule optimizations, and persist task handles so work survives restarts [5].
5. **Pin protocol versions and plan migrations.** MCP now has a 12-month deprecation window [5]. Record which revision each server implements and test against the next one before it becomes mandatory.
6. **Instrument end to end.** Propagate trace identifiers from the agent through the gateway to each server so that a wrong answer or a failed action can be traced to the exact call.

6.2 Risks and limitations

- **Untrusted tools and servers.** A third-party MCP server is code running with access to the firm's data. Install only from known publishers, review tool descriptions for hidden instructions, and pin versions.
- **Over-broad exposure.** The ease of adding servers invites exposing more than a task needs. Every exposed tool widens the space for prompt-injection attacks, which the companion security paper addresses.
- **Write-path immaturity.** Reading from ERP and MES through MCP is well understood; production-grade write-back is less mature, which is another reason to write through drafts and approvals.
- **Standards still moving.** Both protocols changed materially in 2026. The vendor-neutral governance now in place lowers lock-in risk [6, 10], but integrators should expect further revisions.

6.3 A staged adoption path for mid-sized firms

Table 5 sets out four stages, each adding one dimension of capability.

Table 5. A staged adoption path for agent integration in mid-sized firms.

Stage	Scope	Exit criterion
1 Read-only pilot	One agent, two or three read-only servers (for example ERP orders and historian trends) behind a gateway	Users trust the answers; audit log complete
2 Staged writes	Add draft-creating tools that feed existing approvals	Approval rate and error rate tracked for 60 days
3 Multiple agents	Add maintenance or scheduling agents on the same gateway and servers	Connector count grows as N + M, not N × M
4 Cross-boundary	Use A2A with a supplier or customer agent for one bounded process	Contract terms and agent-card trust in place

Author's proposal; exit criteria are suggested defaults.

The stages deliberately add capability one dimension at a time (read, then write, then more agents, then outside parties), so each step is reversible and the evidence from one stage justifies the next.

The choice of first systems matters as much as the choice of first agent. Good candidates are systems that already have a documented interface, a clear owner and data that people trust: in most mid-sized manufacturers this means the ERP's order and inventory data and the historian, rather than spreadsheets or a lightly used quality database. The first pilot should also include the gateway, even when only one agent and two servers are involved. Adding governance after several agents are in use is far harder than starting with it, because each later agent then has to be moved onto new credentials and a new audit path.

7. Conclusion

Integration, not model capability, now decides whether an agent project reaches production. MCP and A2A give firms two complementary open standards: MCP for connecting an agent to tools and data, and A2A for delegating work between agents. Both moved to Linux Foundation governance and reached enterprise-ready revisions in 2026, which lowers the risk of building on them [5, 6, 8, 10]. The standards alone do not make integration safe. The value comes from the architecture around them: one governed gateway, narrow and well-described tools, writes that pass through existing approvals, and a firm line that keeps agents out of the control layer. Firms that adopt in stages, starting with read-only access to one or two systems, can turn the N × M integration burden into an N + M one while keeping each step reversible.

References

1. Anthropic. (2025, December 9). How enterprises are building AI agents in 2026 [Summary of the 2026 State of AI Agents Report, with Material]. claude.com/resources/articles/how-enterprises-are-building-ai-agents-in-2026
2. Outlook Business. (2025, June). Over 40% of agentic AI projects will be scrapped by 2027, says Gartner. *Outlook Business*. www.outlookbusiness.com/amp/story/deeptech/artificial-intelligence/over-40-of-ag
3. Morris, L. (2025). Gartner predicts most agentic AI projects will fail by 2027. *The National CIO Review*. nationalcioreview.com/articles-insights/extra-bytes/gartner-predicts-most-agenti

4. Salesforce. (2025). Integration key as 93% of IT leaders turn to AI agents [MuleSoft 2025 Connectivity Benchmark Report announcement]. www.salesforce.com/news/stories/connectivity-report-announcement-2025/
5. Claburn, T. (2026, July 29). MCP gets an enterprise makeover. *The Register*. www.theregister.com/ai-and-ml/2026/07/29/mcp-gets-an-enterprise-makeover/5280027
6. MacQueen, A. (2025, December 9). Anthropic, OpenAI, and Block donate AI agent projects to new Linux Foundation body. *diginomica*. diginomica.com/anthropic-openai-and-block-donate-ai-agent-projects-new-linux-fou
7. Agentic AI Foundation. (2026). Model Context Protocol specification, revision 2026-07-28. Retrieved November 12, 2026. modelcontextprotocol.io/specification/2026-07-28
8. Agentic AI Foundation. (2026). MCP 2026 changes the rules [Blog post]. aaif.io/blog/mcp-2026-changes-the-rules
9. Model Context Protocol. (2026). Security best practices. Retrieved November 12, 2026. modelcontextprotocol.io/specification/latest/basic/security_best_practices
10. The Linux Foundation. (2026, April 9). A2A Protocol surpasses 150 organizations, lands in major cloud platforms, and sees enterprise production use in first year [Press release]. www.linuxfoundation.org/press/a2a-protocol-surpasses-150-organizations-lands-in-
11. SAP. (2026). SAP Sapphire 2026 innovation news guide. www.sap.com/topics/events/sapphire/innovation-news-guide-2026
12. Novotek. (2026). Model Context Protocol (MCP): How to connect AI to OT systems. www.novotek.com/co-uk/news/model-context-protocol-mcp-how-to-connect-ai-to-ot-sy
13. Shinde, S. (2026, January). Agentic AI reads OPC UA servers so you don't have to. FlowFuse. flowfuse.com/blog/2026/01/eliminate-opc-ua-bottleneck-ai-agents/
14. eSEOspace. (2026, November). MCP for manufacturing. eseospace.com/blog/mcp-for-manufacturing/

How to cite this article

Lee, Q. F. (2026). Agent Interoperability Standards: The Model Context Protocol, Agent-to-Agent Protocols and Integration with ERP, CRM and Plant Systems. *Chmltech Online Journal*, *105*(4). https://www.chmltech.com/post/agent-interoperability-standards-mcp-a2a-erp-crm-plant-systems